



# UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

*Universidad decana de América*

# Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional de Ingeniería de Sistemas.

# Estructura de Datos

Prof. G. A. Salinas

# Semana 13

# Grafos

Ciclo 2026-1

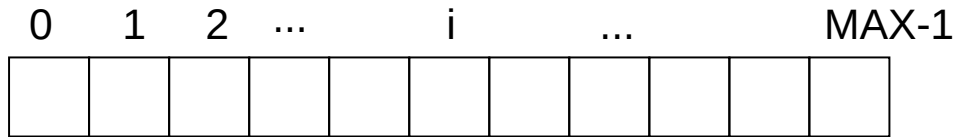
# Agenda

1. Introducción
2. Definición
3. Tipos de grafos
4. Representación
5. Operaciones básicas
6. Aplicaciones
7. Recorrido en profundidad
8. Recorrido en anchura
9. Resumen
10. Referencias

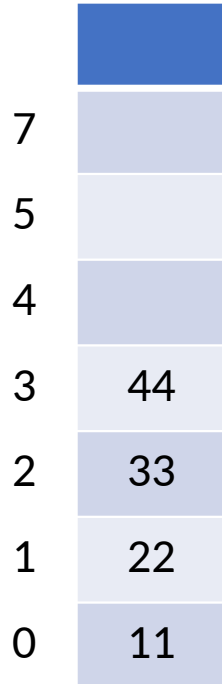
# 1. Introducción

- **Estucturas lineales:** Arreglos, listas enlazadas, pilas y colas

Arreglos



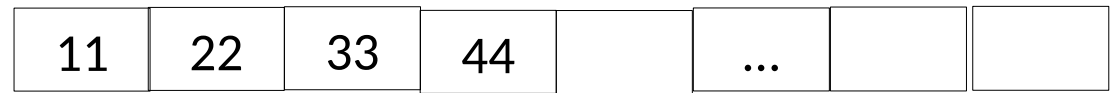
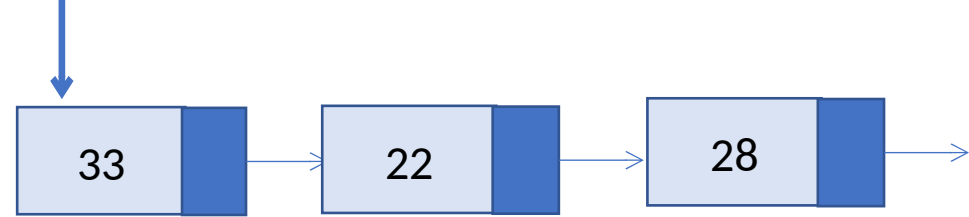
MAX



← Tope

Pilas

Lista



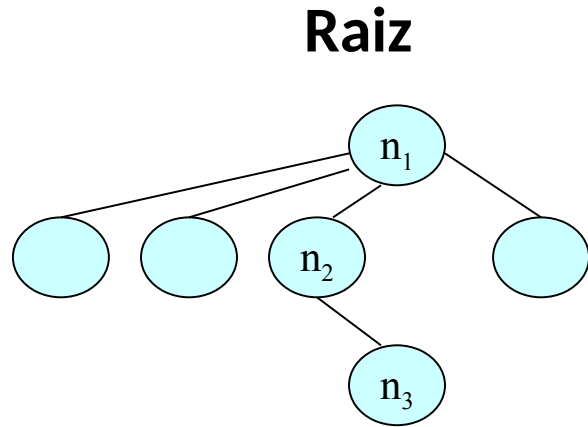
↑  
Frente

↑  
Final

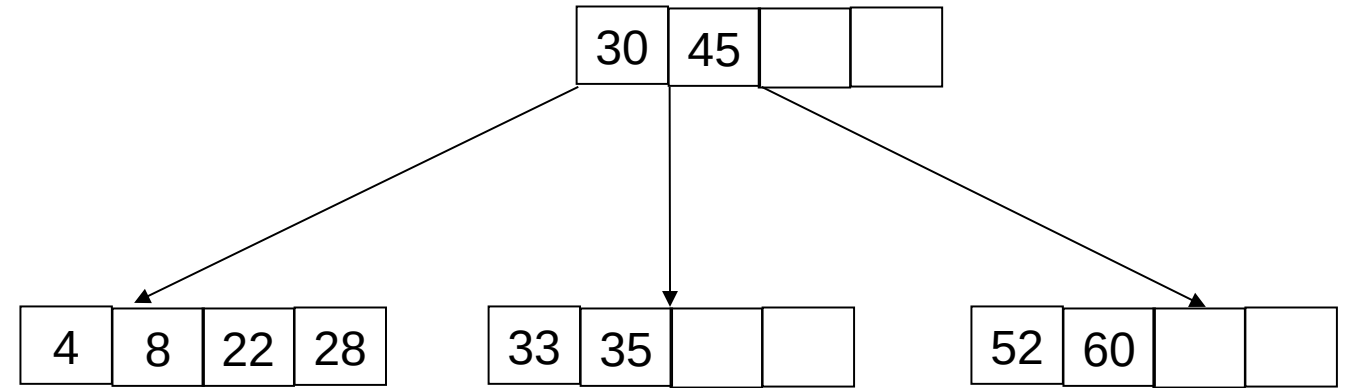
Colas

# 1. Introducción

- **Estructuras gerárquicas. N-arios, binarios**

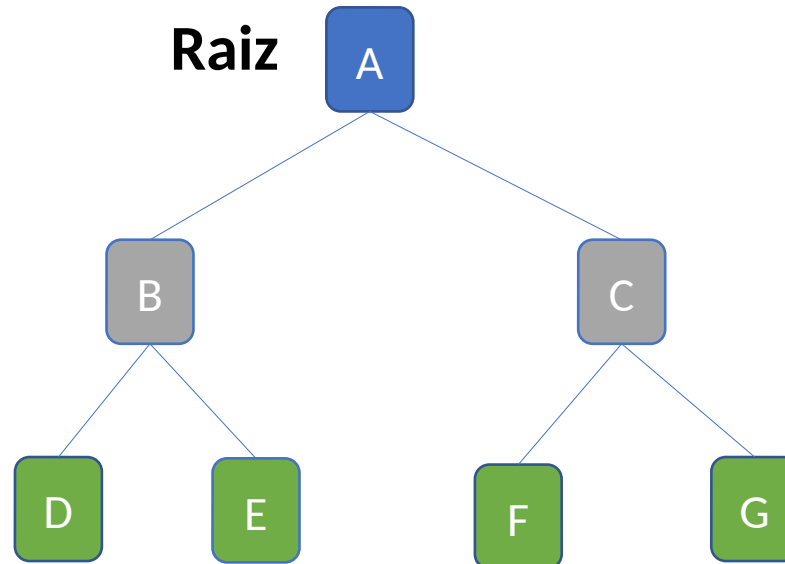


Arboles  
N-arios



Arboles B

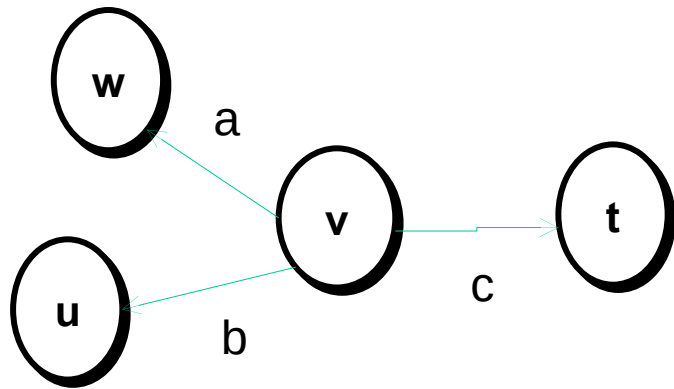
Arboles  
Binarios



# 1. Introducción

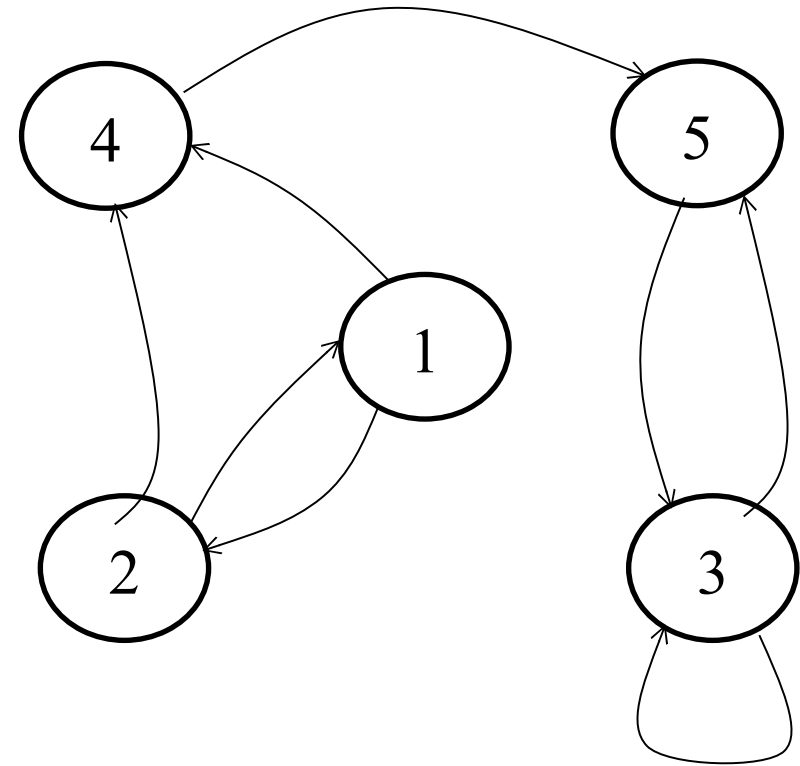
- **Estructuras no lineales.** Grafos:  $G = (V, A)$

Es una forma de representar relaciones entre pares de elementos.



$$V = \{u, w, v, t\}$$

$$A = \{a, b, c\}$$

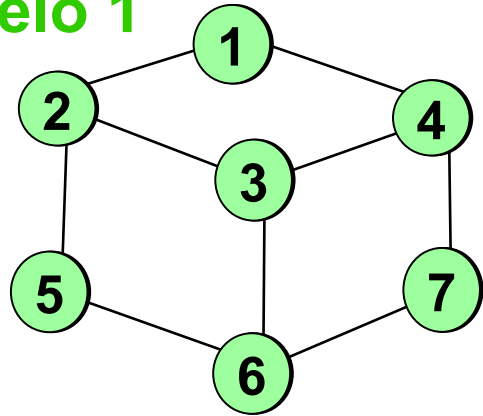




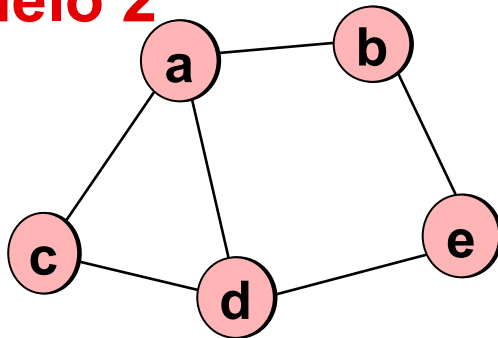
# 1. Ejemplos.

- **Ejemplo:** Grafo asociado a un dibujo de líneas.

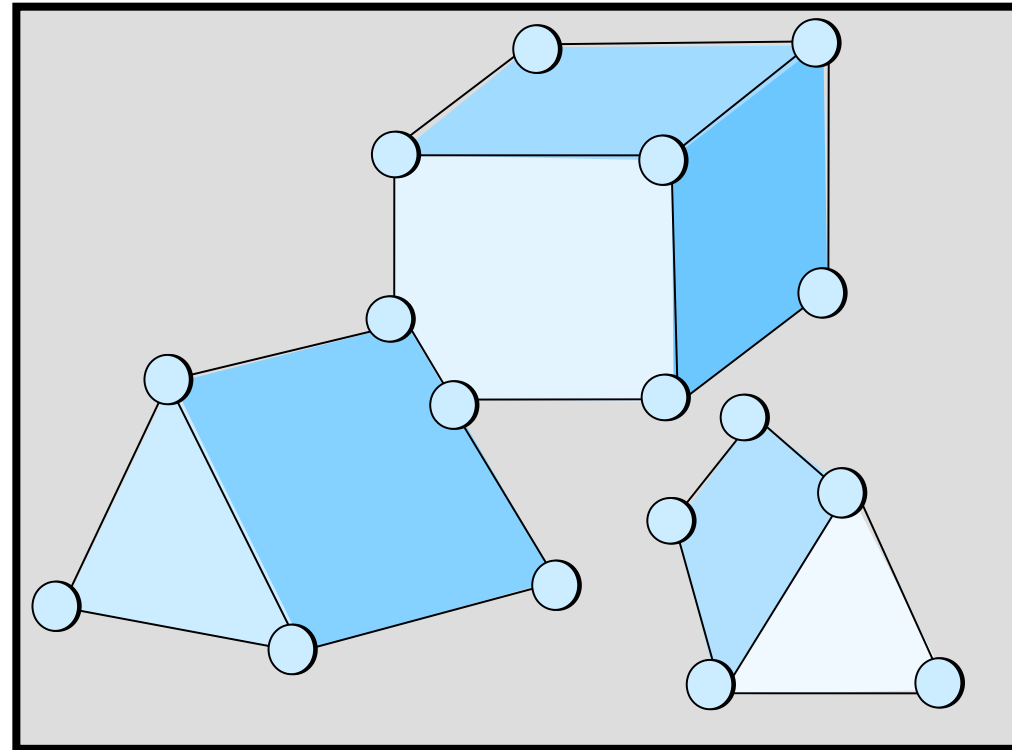
**Modelo 1**



**Modelo 2**

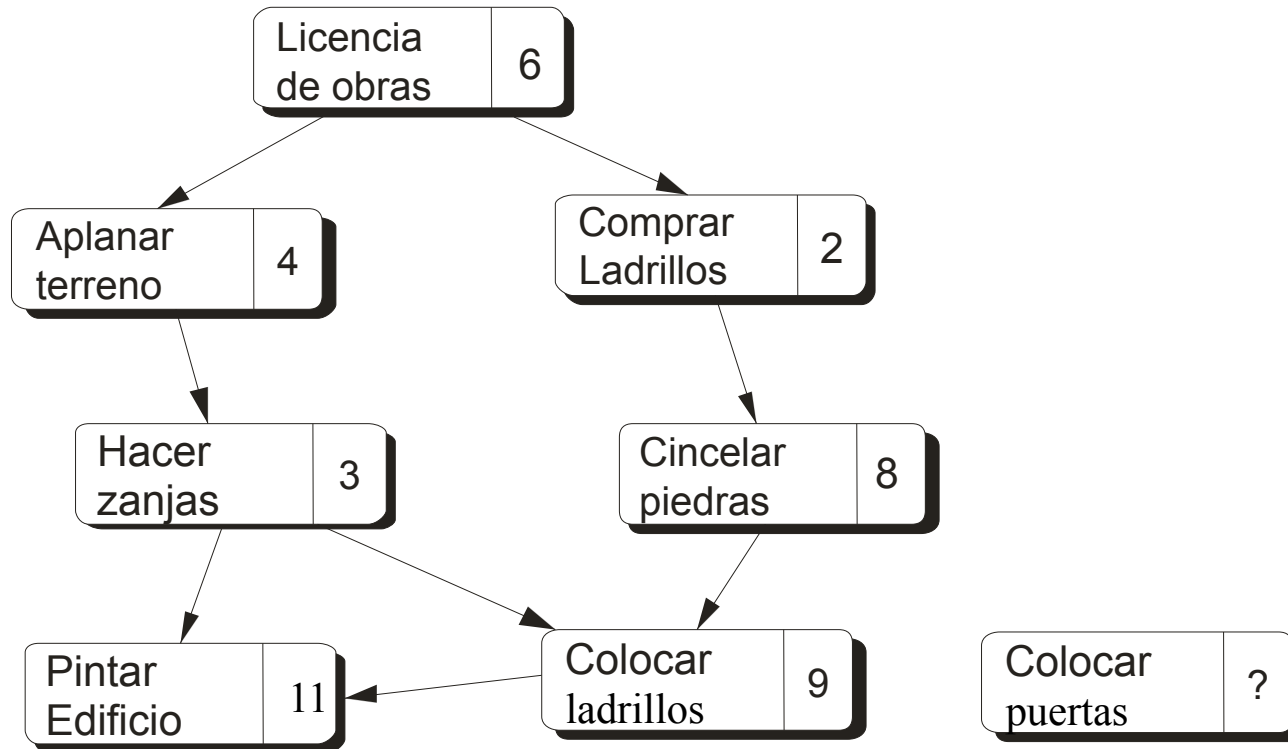


**Escena**



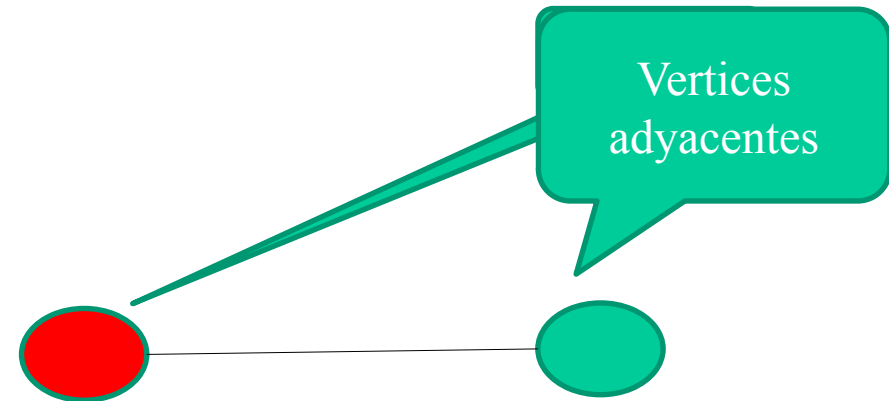
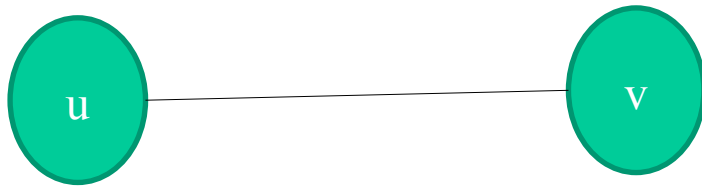
# 1. Ejemplos de grafos.

- **Ejemplo:** Grafo de planificación de tareas.



## 2. Definición

- **Un grafo  $G$**  agrupa entes físicos o conceptuales, es el par  $G = (V, A)$ , donde  $V$  es un conjunto no vacío de **vértices** o **nodos** (entes) y  $A$  es un conjunto de **aristas** o **arcos**, que conectan pares de vértices distintos (relaciones). }
- Figura: Arista, **vertices**, **incidencia**



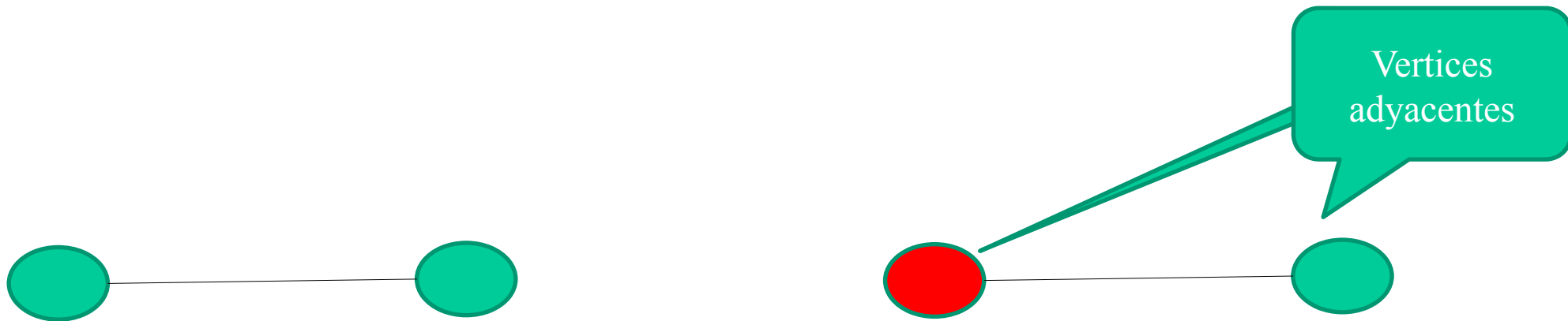
**Arista incidente en el vertice**

## 2. Definición

- **Un grafo**

- **Los Grafos.** Enfatizan las relaciones complejas de conexión entre aristas de datos. Los arboles son un caso simple de grafos.
- **Los grafos permiten problemas como.** Menor distancia o costo entre puntos búsquedas a través de enlaces en la web; redes eléctricas; cableado de tarjetas impresas; planeación de tareas siguiendo trayectorias;

- **Figura: Arista, vertices, incidencia**



Arista incidente en el vertice

## 2. Definición

- **Grado de incidencia del vértice.** Es el numero de aristas que son incidentes en ese vértice.
- **Adyacencia.** En grafos simples se aceptan solo un enlace entre un par de vértices diferentes, no se aceptan lazos o aristas adyacentes en el mismo vértice, ni aristas en paralelo.
- **Trayectoria o camino.** Secuencia de aristas entre vértices adyacentes.
- **Trayectoria simple.** Los vértices y las aristas están presentes solo una vez.
- **Ciclo o circuito.** Trayectoria simple, salvo que el vértice inicial y final son uno solo.
- **Longitud de la trayectoria.** Es el numero de aristas que la componen
- **Grado de un grafo.** Si existe trayectoria entre cualquier par de vértices.

## 2. Definición

- **Árbol de un grafo.** Subgrafo conectado sin circuitos.
- **Densidad.** Es el promedio de los grados de incidencia de los vértices. Si sumamos los grados de la incidencia se tendrá el valor de  $2A$  (2 veces cada arista). Por tanto la densidad es:  $2A/V$ .
  - Se dice que un grafo es denso si su densidad es proporcional a  $V$ . Esto implica que el numero de aristas  $A$  será proporcional a  $V^2$ . En caso contrario es un grafo liviano o sparce.
  - El concepto de densidad tiene relación con el tipo de algoritmo y la estructura de datos que se empleara para representar el gafo. Complejidad típicas son:  $V^2$  , y  $A \log A$ , la primera es mas adecuada a grafos densos y la segunda a grafos livianos.

## 2. Definición

- Un grafo  $G(V, A)$  es un conjunto no vacío, donde:
  - $V = \{v_1, v_2, \dots, v_M\}$  conjunto de nodos o vértices.
  - $A = \{a_1, a_2, \dots, a_K\}$  es el conjunto de aristas dirigidos (par ordenado de Vértices:  $(u, v)$ )
- Los bucles tienden a hacer mas complejas los algoritmos de grafos/.
- Un grafo simple es un grafo que no tiene bucles
- Un grafo con  $|V| = n$ , cada vertice podria tener un maximo:  
 $0 \leq |A| \leq n(n-1)$

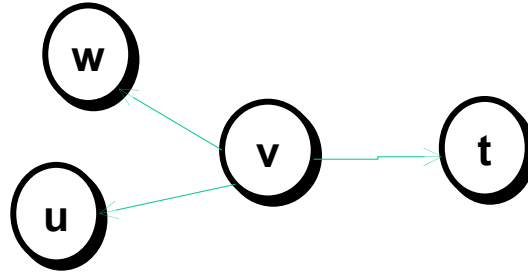
## 2. Definición

- Un grafo con  $|V| = n$ , cada vertice podria tener un maximo:  
 $0 \leq |A| \leq n(n-1)$
- Numero maximo de aristas en un grafo simple No dirigido: Si  $|V| = n$   
cada vertice podria tener  $(n - 1)$  aristas:  $0 \leq |A| \leq n(n-1) / 2$
- Un grafo es denso si el numero de aristas es cercano a su numero maximo posible:  $n(n-1)$  o  $n(n-1)/2$  :  $n^2$
- Un grafo es escaso si el numero de aristas es cercano al numero de de sus vertices:  $n$
- Conocer si un grafo es denso o poco denso nos permitira elegir la implementacion adecuada: matriz o lista de adyacencia



## 2. Definición

- Un camino de un grafo es una secuencia de vértices tal que exista una arista (camino) entre cada vertice y el siguiente: ejemplo:  $\{w, v, t\}$

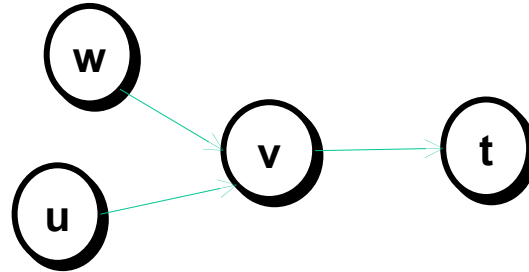


## 2. Definiciones.

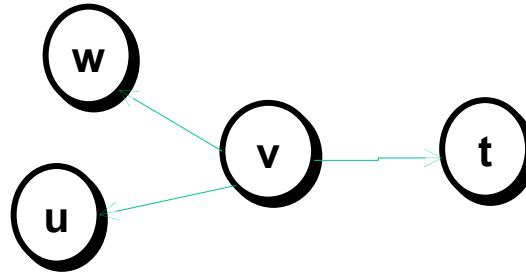
- **Grafos dirigidos (o digrafos).** Arcos con dirección

- $(w, v) \neq (v, w)$

- **Grado de entrada** de un vértice  $v$ : numero de arcos cuyo vértice final es  $v$ .



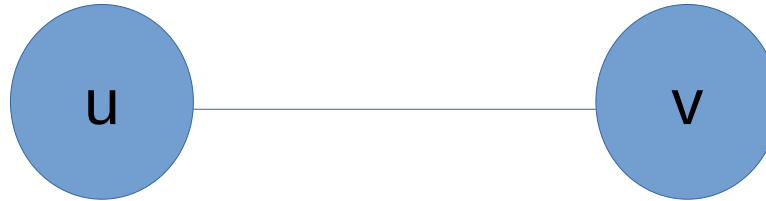
- **Grado de salida** de un vértice  $v$ : numero de arcos cuyo vértice inicial es  $v$ .



### 3. Tipos

- **Tipo de grafos:**

- **Grafos No dirigido.** Las aristas son bidireccionales no tienen sentido orientado..



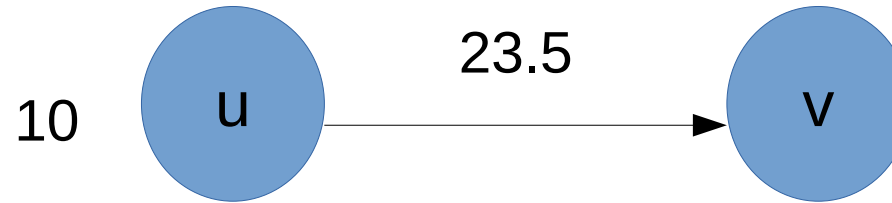
- **Grafo dirigido (Digrafo)** Las aristas tiene tiene sentido orientado (flechas)



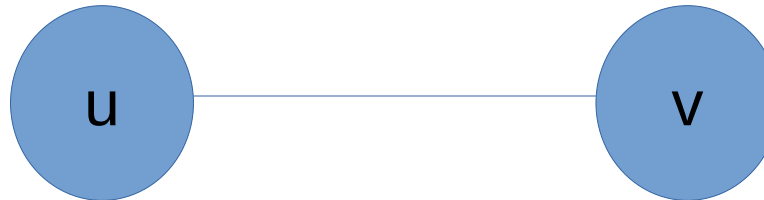
### 3. Tipos

- **Tipo de grafos:**

- **Grafos ponderados.** Las aristas  $A(u, v)$  o los vértices tienen asociado un costo o una distancia.



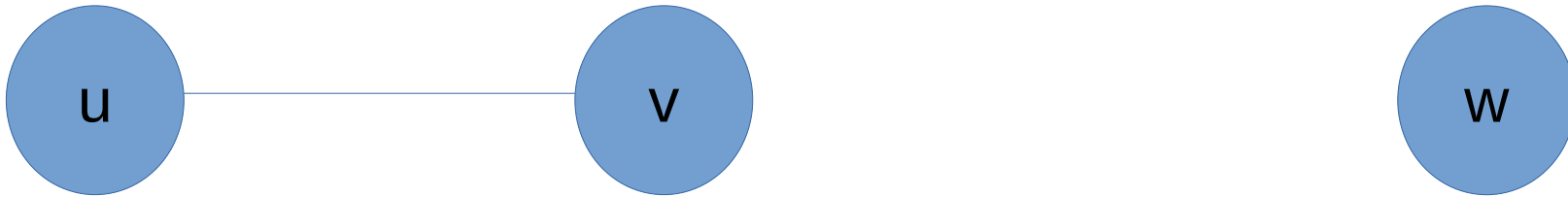
- **Grafo no ponderado.** Todas las aristas tienen el mismo valor o cero.



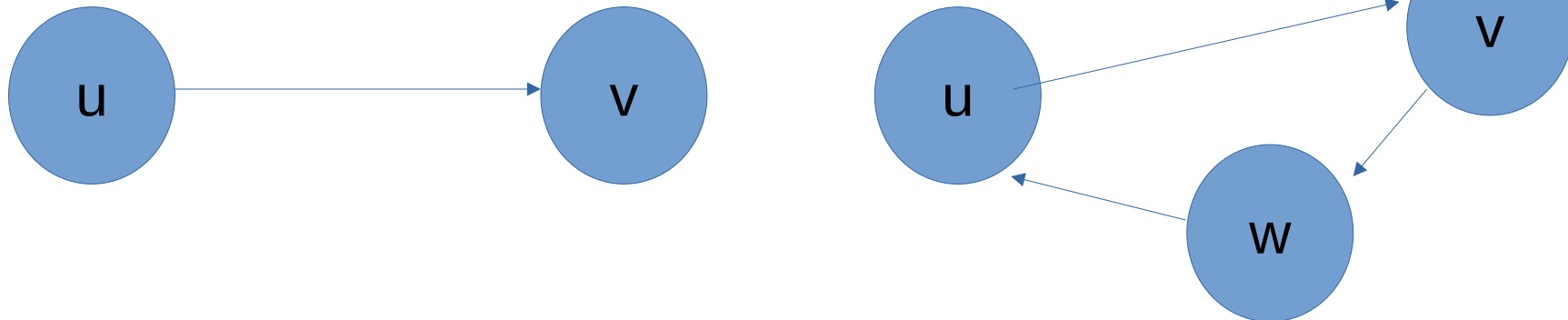
### 3. Tipos

- **Tipo de grafos:**

- **Grafos conexo.** Existe al menos un camino entre cualquier par de vertices



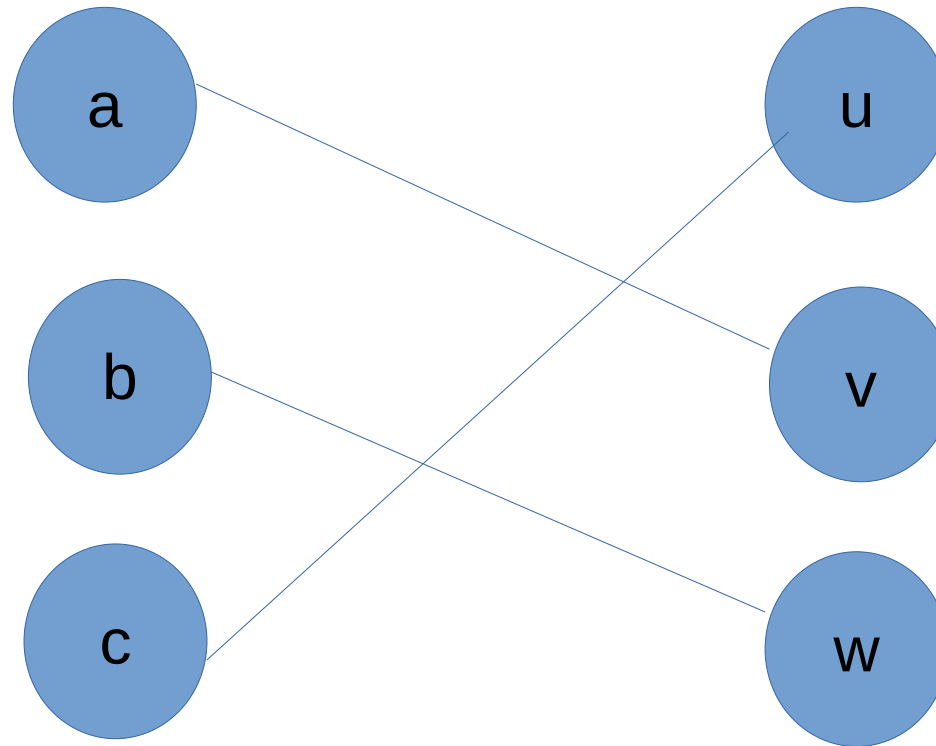
- **Grafo Aciclico/ciclico.** Un ciclo inicia en un vórtice y termina en el mismo vértice



### 3. Tipos

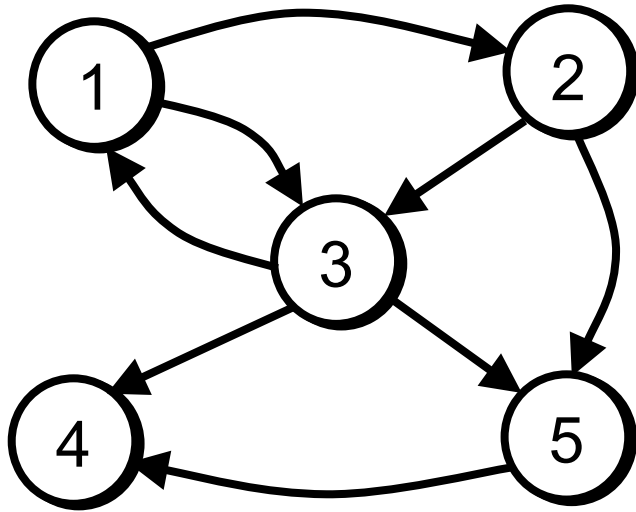
#### Tipo de grafos:

- **Grafos bipartito.** Vértices de grupos independientes. Las artistas conectan un nodo de un grupo con uno de otro grupo. No existen conexiones con nodos del mismo grupo.



# 4. Representación de Grafos

- **Representación de grafos:**
  - Representación del conjunto de nodos,  $V$ .
  - Representación del conjunto de aristas,  $A$ .



$$V = \{1, 2, 3, 4, 5\}$$

$$A = \{(1, 2) (1, 3) (2, 3) (2, 5) (3, 1) (3, 4) (3, 5) (5, 4)\}$$

- **Ojo:** las aristas son relaciones “muchos a muchos” entre nodos...

# 4. Representación de Grafos

**Matriz de Adyacencia.** (representación conjunto aristas)

Usa un arreglo de  $N \times N$  ( $N$  número de nodos) en el que  $a[v_i, v_j]$  es igual a:

1 : Si existe una arista desde el vértice  $v_i$  al vértice  $v_j$

0 : En otro Caso.

Las columnas y las filas de la matriz representan los nodos del grafo. Si existe un arco desde  $i$  a  $j$  (esto es, el nodo  $i$  es adyacente al nodo  $j$ )

Si el grafo es ponderado, el costo o peso asociado al arco de  $i$  a  $j$  se almacena, si no existe el arco, se introduce 0.

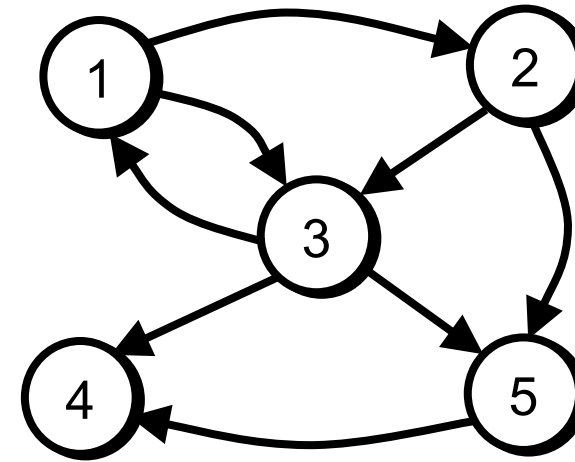
- Si el grafo es no dirigido, la matriz es simétrica  $M(i,j) = M(j,i)$ .
- Es conveniente la representación para grafos densos.



# 4. Representación de Grafos

## Matriz de Adyacencia

|   | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 0 | 0 |
| 2 | 0 | 0 | 1 | 0 | 1 |
| 3 | 1 | 0 | 0 | 1 | 1 |
| 4 | 0 | 0 | 0 | 0 | 0 |
| 5 | 0 | 0 | 0 | 1 | 0 |



$$V = \{1, 2, 3, 4, 5\}$$

$$A = \{(1, 2) (1, 3) (2, 3) (2, 5) (3, 1) (3, 4) (3, 5) (5, 4)\}$$

En los grafos no dirigidos, la matriz de adyacencia va ser simetrica  $M_{i,j} = M_{j,i}$

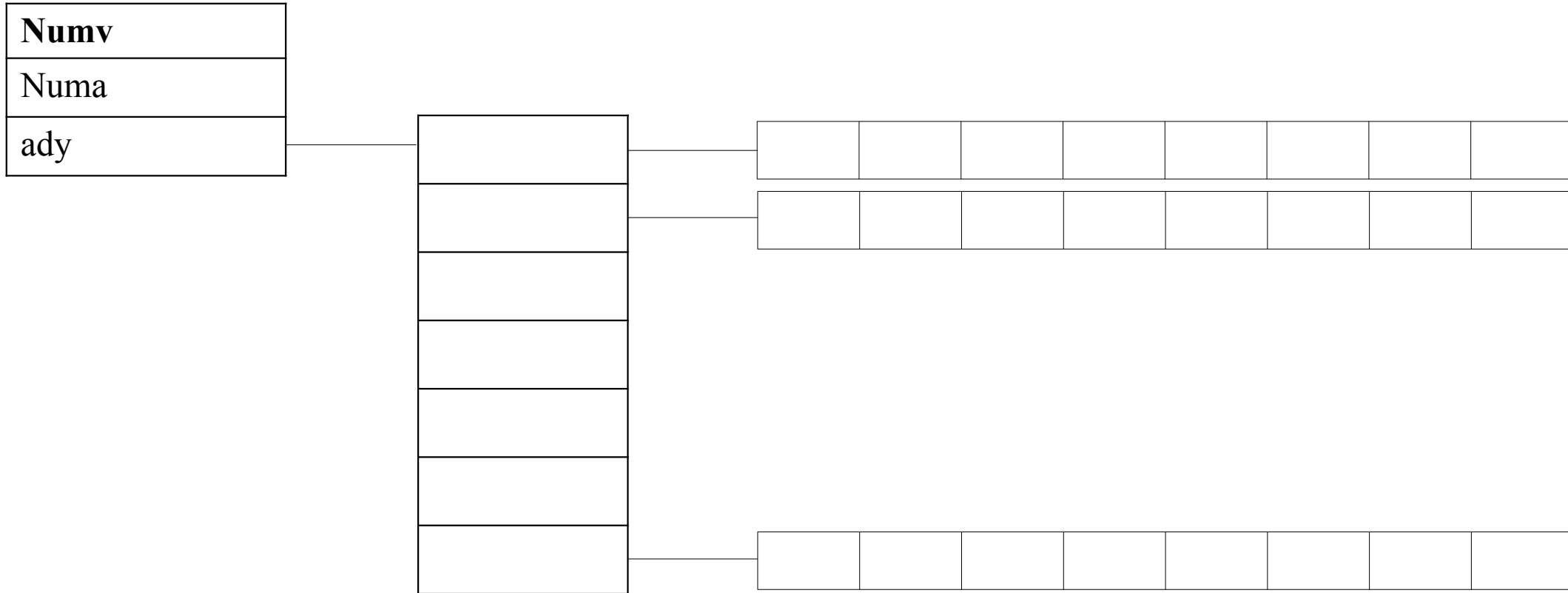
# 4. Representación de Grafos

**Matriz de Adyacencia.** (representación conjunto aristas)

```
struct GRAFO{
    int numv;
    int numa;
    int **mady;
};
GRAFO g=malloc(sizeof *g);           //Crea la cabecera del gafo
int **ma =malloc(nfil*sizeof(int*)); //Crea vector de apuntadores de nfil filas.
g->ady=ma;
for(i=0;i<nfil;i=i+1) {
    ma[i]=malloc(ncol*sizeof(int));
}
```

# 4. Representación de Grafos

**Matriz de Adyacencia.** (representación conjunto aristas)



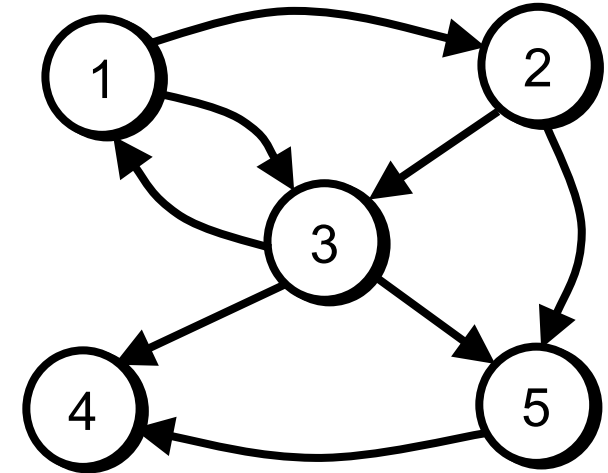
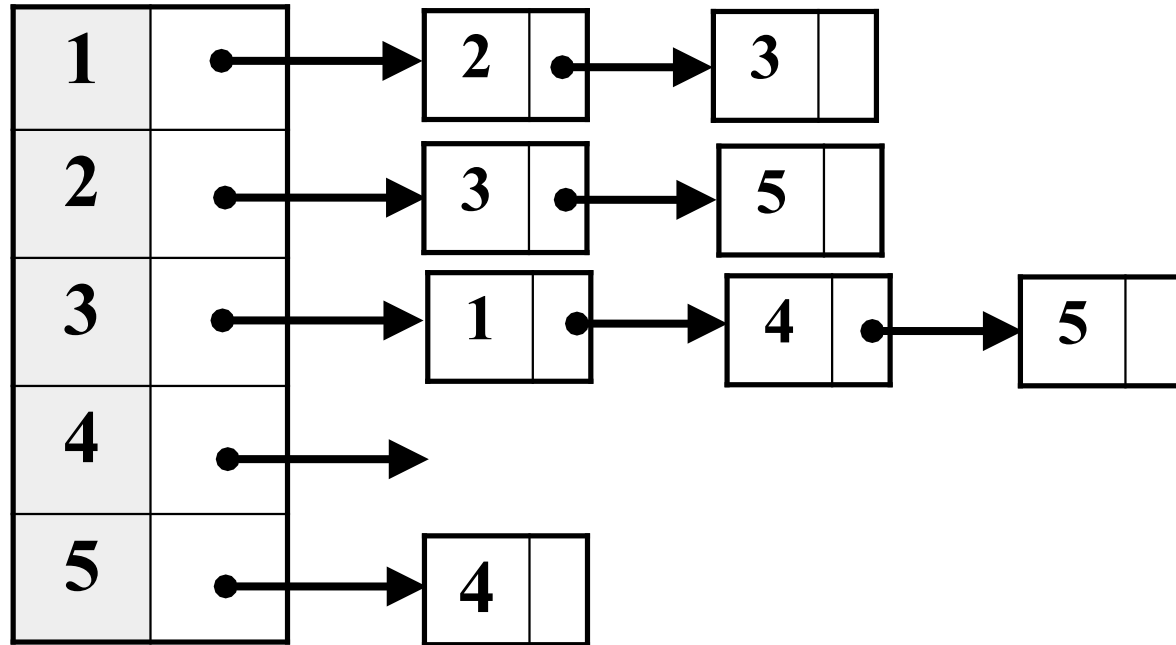
## 4. Representación de Grafos

### **Listas de Adyacencia.**

Cada vértice tiene una lista de vertices los cuales son adyacentes a el. Esto causa redundancia en un grafo no dirigido (ya que a existe en la lista de adyacencia de B y viceversa) pero las búsquedas son mas rápidas, al costo de almacenamiento extra.

# 4. Representación de Grafos

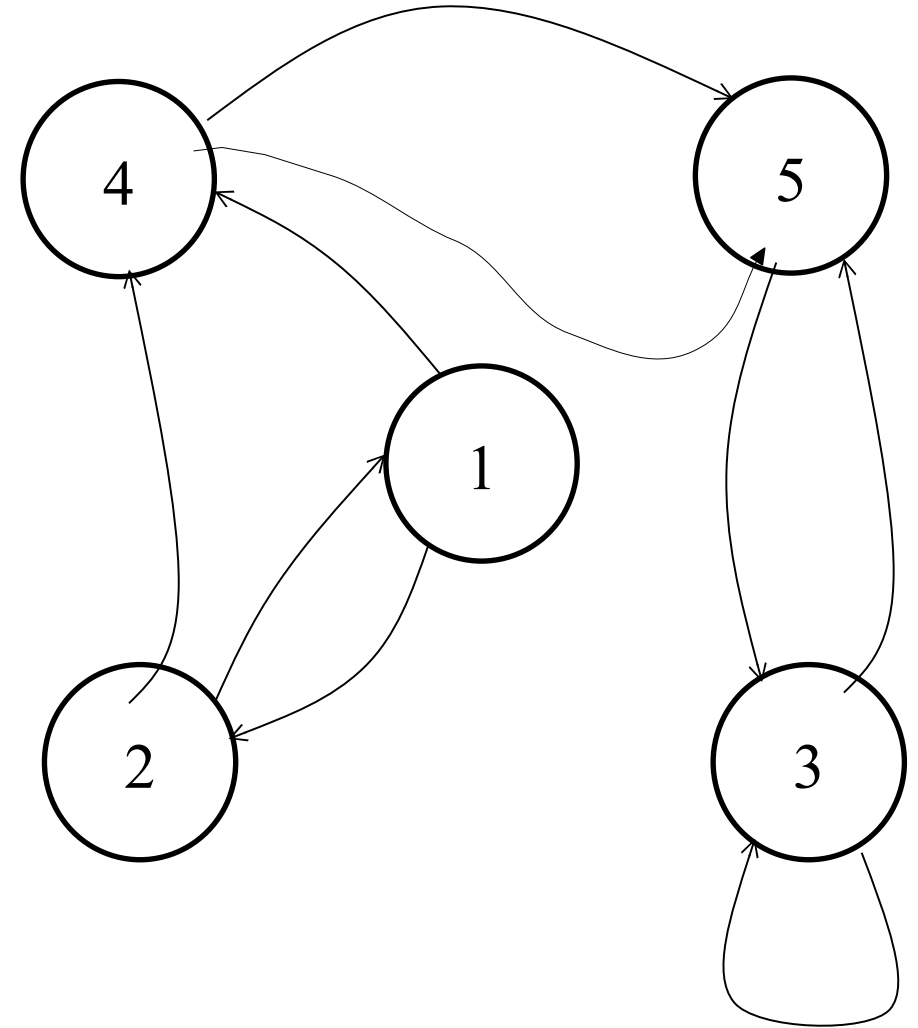
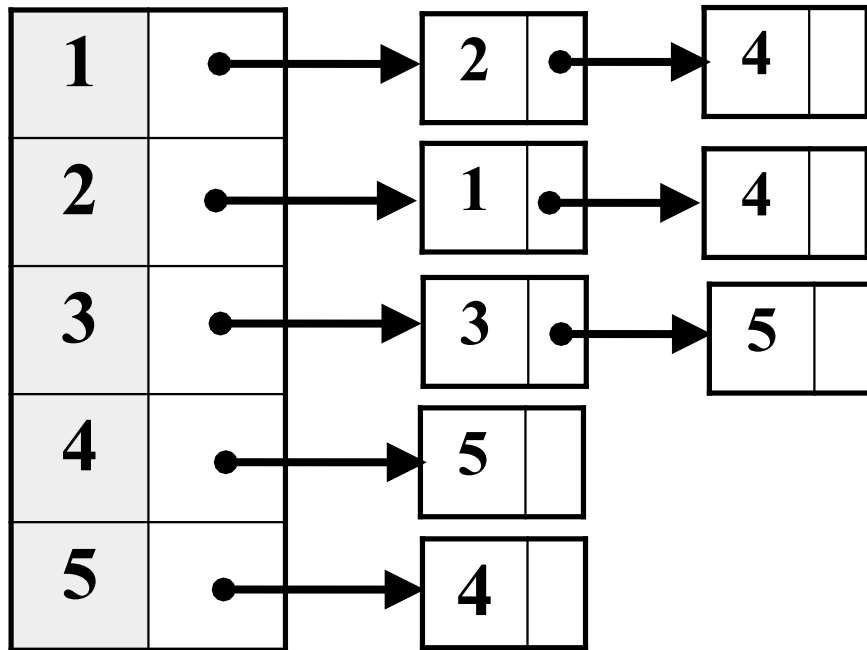
## 4.2 Lista de Adyacencia



# 4. Representación de Grafos

## 4.2 Lista de Adyacencia

Los sucesores de nodo se enlazan en una misma lista enlazada, un asociada a cada nodo de un Array de listas o lista de listas.



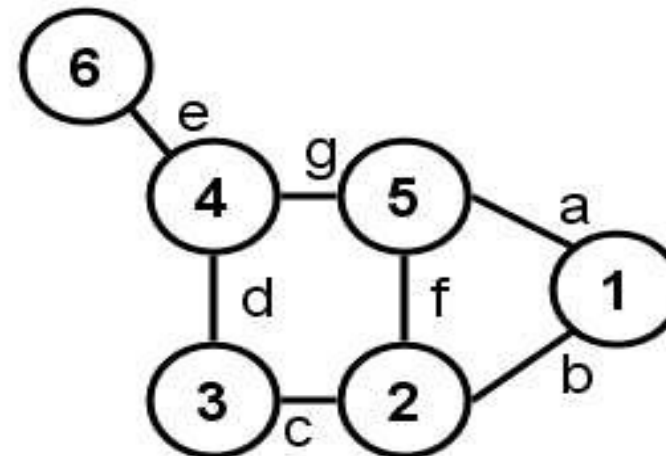
# 4. Representación de Grafos

## Matriz de Incidencia.

De un grafo no dirigido  $G(V, A)$  es una matriz  $|V| \times |A|$ . Con  $i(j, k) = 0$ , cuando el arco  $k$  no incide en nodo  $j$  (no sale/llega). Con  $i(j, k) = 1, 2$ , cuando el arco  $k$  incide una o dos veces en el nodo  $j$ .

El grafo esta representado por una matriz de  $A$  aristas por  $V$  vértices donde  $[arista, vértice]$  contiene la información de la arista conectado o no conectado.

|   | a | b | c | d | e | f | g |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 0 | 0 | 0 | 1 |
| 2 | 0 | 1 | 1 | 0 | 0 | 1 | 0 |
| 3 | 0 | 0 | 1 | 1 | 0 | 0 | 0 |
| 4 | 0 | 0 | 0 | 1 | 1 | 0 | 1 |
| 5 | 1 | 0 | 0 | 0 | 0 | 1 | 1 |
| 6 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

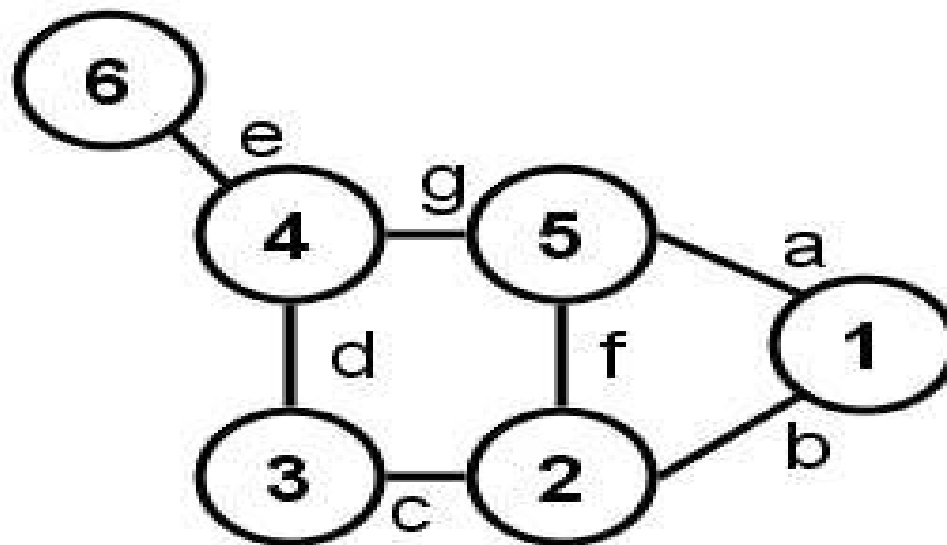


# 4. Representación de Grafos

## 4.2 Lista de Incidencia.

El grafo esta representado por un arreglo (vector) de pares (ordenados, si el grafo es dirigido), donde cada par representa una de las aristas

- Lista: ((6,4) (4,5) (4,3) (3,2) (5,2) (5,1) (2,1) )





# 4. Representación de Grafos

## Costo.

### 1. Matriz de adyacencia

- Memoria:  $O(|V|^2)$  .
- Costo de saber si hay un arco  $(v,w)$ :  $O(1)$ .

### 2. Lista de adyacencia

- Memoria:  $O(|V|+|E|)$
- Costo de saber si hay un arco  $(v,w)$ :  $O(|E|)$ .

### 3. Matriz de incidencia

- Memoria:  $O(|V| \cdot |E|)$
- Costo de saber si hay un arco  $(v,w)$ :

# 5. Operaciones

- Operaciones Básicas
  - Creación
  - Inserción
  - Eliminación
  - Edición
- Recorrido en profundidad (DFS)
- Recorrido en anchura (BFS)

# 5. Recorrido de Grafos

**Existen dos recorridos:**

- Recorrido en profundidad (DFS)
- Recorrido en anchura (BFS)

# 6. Aplicaciones

## **Aplicaciones de busqueda en profundidad (DFS Depth First Search)**

- **Deteccion de ciclos en un grafo.**
- **Busqueda de rutas.** Para busqueda entre dos vertices.
- **Ordenamiento topologico.** Para programar ordenamiento a partir de las dependencias entre ellas (Prerrequisitos en un malla curricular)
- **Comprobar si un grafo es bipartito.** Para coleroreado de mapas.
- **Compenentes fuertemente conectados de un grafo.** una ruta hacia todos los vertices. (Subconjunto vertices donde cada par de nodos es mutumente alcanzable)
- Resolver rompecabezas con una sola solucion. Laberintos
- Generacion de laberintos

# 6. Aplicaciones

## **Ventajas de búsqueda en profundidad (DFS)**

- Recorre por una rama lo mas profundo posible antes de retroceder, utiliza una pila
- El requerimiento de memoria es lineal con respecto a la búsqueda en amplitud(BFS) que requiere de mas espacio.
- La complejidad temporal de una búsqueda profundidad es practicamente limitada por el tiempo y no por el espacio.
- Encuentra la solucion sin explorar mucho en tiempo y en espacio.
- Requiere menos memoria, se almacenan solo los nodos de la ruta actual.
- Ideal para deteccion de ciclo y ordenamiento topologico.

# 6. Aplicaciones

## **Desventajas de búsqueda en profundidad (DFS Depth First Search)**

- Existe la posibilidad de que se extienda por el camino de mas a la izquierda indefinidamente incluso en un grafo finito y puede generar una solucion infinita.
- No garantiza que encuentre la solucion.
- No garantiza encontrar la solucion minima si hay mas de una solucion.

# 6. Recorrido de Grafos

## **Recorrido en profundidad (DFS).**

DFS(grafo G)

PARA CADA vértice  $u \in V[G]$  HACER

    estado[u]  $\leftarrow$  NO\_VISITADO

    padre[u]  $\leftarrow$  NULO

tiempo  $\leftarrow$  0

PARA CADA vértice  $u \in V[G]$  HACER

    SI estado[u] = NO\_VISITADO ENTONCES

        DFS\_Visitar(u, tiempo)

## 6. Recorrido de Grafos

```
DFS_Visitar(nodo u, int tiempo)
    estado[u] ← VISITADO
    tiempo ← tiempo + 1
    d[u] ← tiempo
    PARA CADA v ∈ Vecinos[u] HACER
        SI estado[v] = NO_VISITADO ENTONCES
            padre[v] ← u
            DFS_Visitar(v, tiempo)
    estado[u] ← TERMINADO
    tiempo ← tiempo + 1
    f[u] ← tiempo
```



# 6. Aplicaciones

## **Aplicaciones de busqueda en amplitud (BFS Breadth First Search)**

- Visita nodos por niveles, utiliza una cola,
- Ruta mas corta y arbol de expansion minimo para grafos no ponderados.
- Arbol de expansion minimo para graficos ponderados
- Redes de punto a punto. Para encontrar todos los nodos vecinos.
- Rastreadores de motores de busqueda.
- Ordenamiento topologico en un grafo aciclico dirigido..
- Sitios web de redes sociales. Sistema de navegacion GPS.
- Encontrar compenentes conexos y deteccion ciclos en grafos no dirigidos
- Procesamiento de imagenes. Rellenado de color o tratamiento de pixeles
- Algoritmo de Ford-Fulkerson es preferible porque reduce la complejidad temporal
- Para encontrar la ruta mas corta de fuente unica de Dijkstra.

# 6. Aplicaciones

## **Ventajas de busqueda en amplitud (BFS)**

- No se quedara atrapado explorando el camino util para siempre.
- Definitivamente la encontrara, si existe una solucion.
- Puede encontrar la minima solucion con menos cantidad de pasos, si hay mas de una solucion.
- Requisitos de almacenamiento bajos, con requerimiento lineal de DFS.
- De facil programacion.

# 6. Aplicaciones

## **Desventajas de búsqueda en amplitud (BFS)**

- Alto requerimiento de espacio de memoria, debe guardar cada nivel el grafo para generar el siguiente y que es proporcional al numero de nodos almacenados.
- En la practica agota la memoria disponible en computadoras tipicas en cuestion de minutos.

# 6. Aplicaciones

## **Recorrido en anchura (BFS).**

BFS(grafo G, nodo\_fuente s)

// recorreremos todos los vértices del grafo inicializándolos a NO\_VISITADO,

// distancia INFINITA y padre de cada nodo NULL

for u  $\in$  V[G] do

    estado[u] = NO\_VISITADO;

    distancia[u] = INFINITO; /\* distancia infinita si el nodo no es alcanzable \*/

    padre[u] = NULL;

F\_For

    estado[s] = VISITADO;

    distancia[s] = 0;

    padre[s] = NULL;

    CrearCola(Q); /\* nos aseguramos que la cola está vacía \*/

    Encolar(Q, s);

## 6. Aplicaciones

```
while !vacía(Q) do
  // extraemos el nodo u de la cola Q y exploramos todos sus nodos adyacentes
  u = extraer(Q);
  for v ∈ adyacencia[u] do
    if estado[v] == NO_VISITADO then
      estado[v] = VISITADO;
      distancia[v] = distancia[u] + 1;
      padre[v] = u;
      Encolar(Q, v);
```

Fif

Ffor

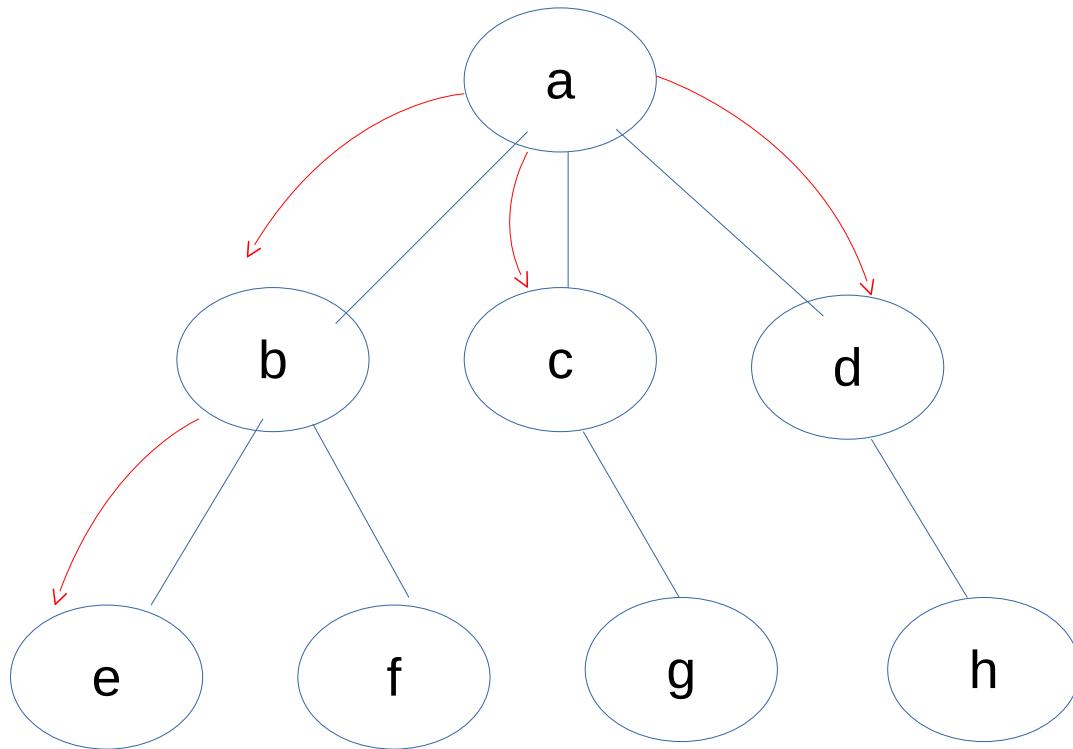
Fwhile

FBfs

[https://es.wikipedia.org/wiki/B%C3%BAsqueda\\_en\\_anchura](https://es.wikipedia.org/wiki/B%C3%BAsqueda_en_anchura)

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}



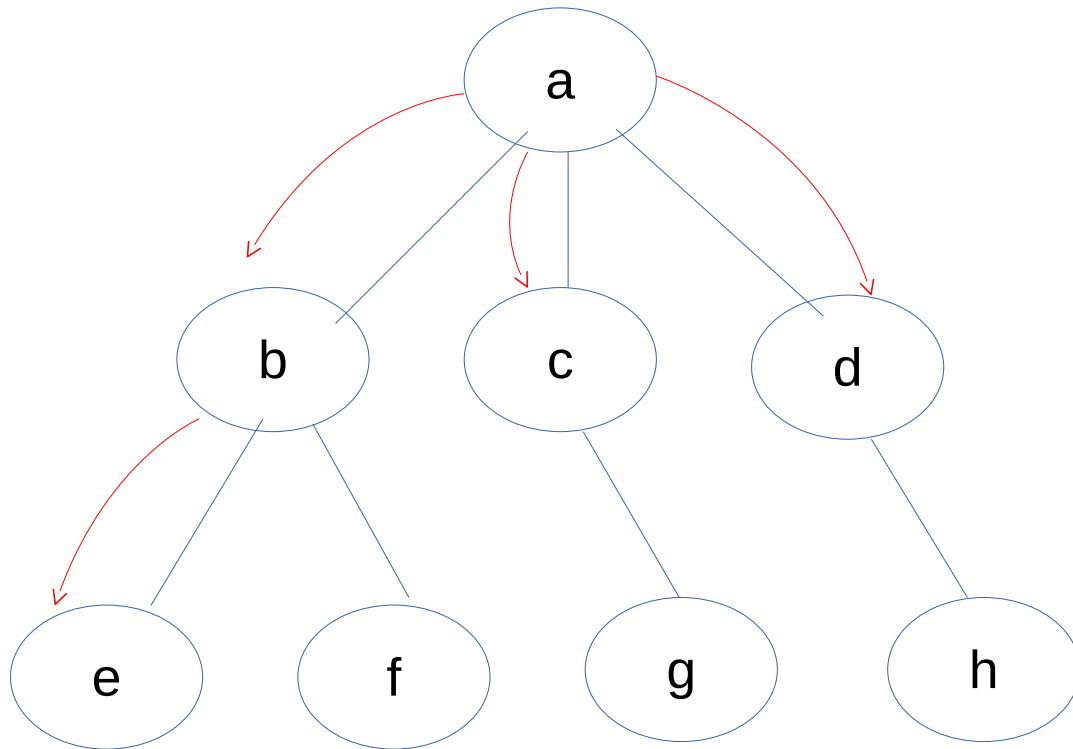
Mientras la cola no este vacia

- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes



# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

|   |   |   |   |   |   |   |  |  |
|---|---|---|---|---|---|---|--|--|
| a | b | c | d | e | f | g |  |  |
|---|---|---|---|---|---|---|--|--|

Mientras la cola no este vacia

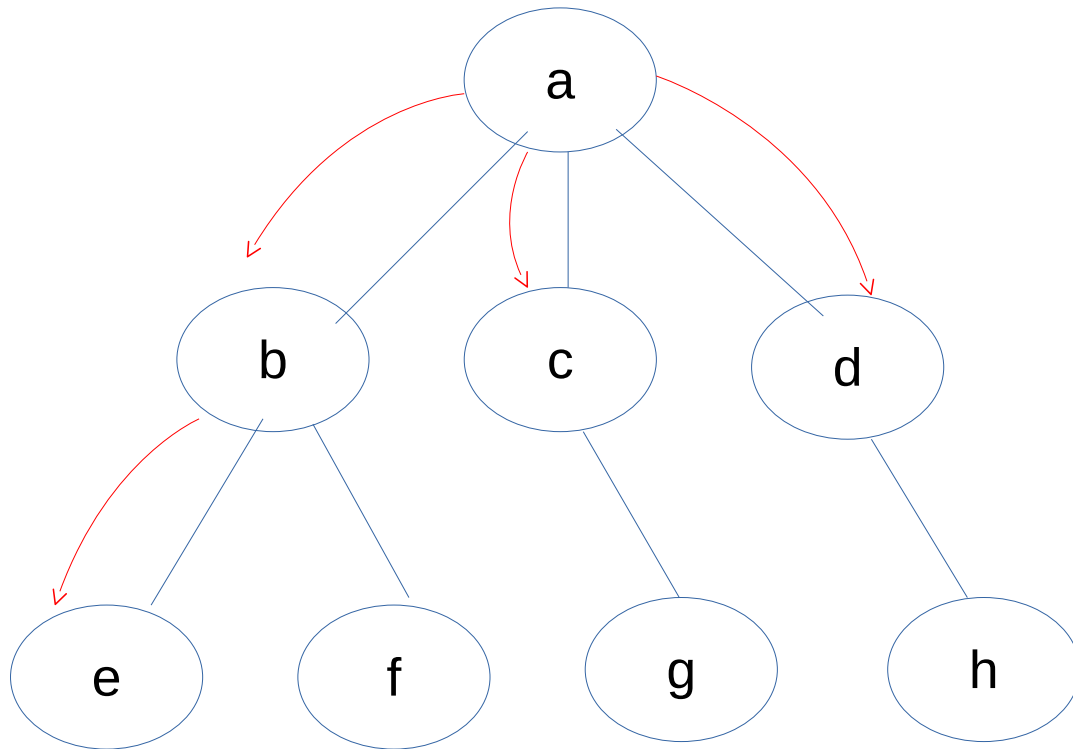
- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

|  |  |  |   |   |   |   |  |  |
|--|--|--|---|---|---|---|--|--|
|  |  |  | d | e | f | g |  |  |
|--|--|--|---|---|---|---|--|--|

|   |   |   |  |  |  |  |  |  |
|---|---|---|--|--|--|--|--|--|
| a | b | c |  |  |  |  |  |  |
|---|---|---|--|--|--|--|--|--|

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

Mientras la cola no este vacia

- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

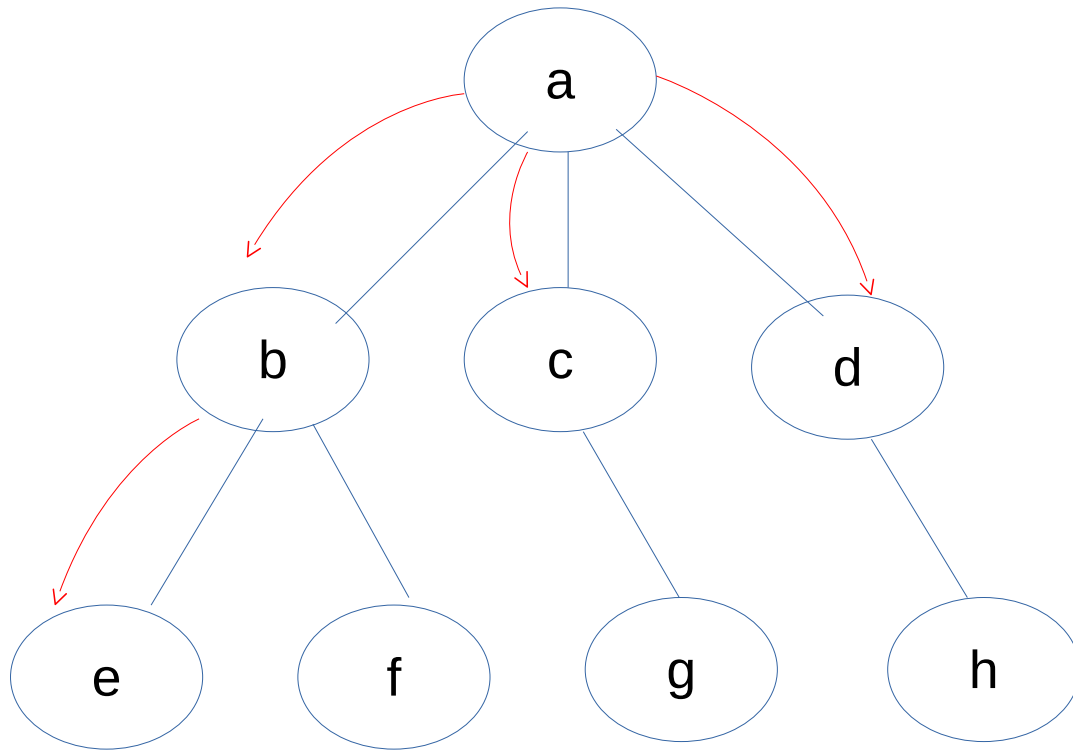
|  |  |  |  |   |   |   |   |  |
|--|--|--|--|---|---|---|---|--|
|  |  |  |  | e | f | g | h |  |
|--|--|--|--|---|---|---|---|--|

|   |   |   |   |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|
| a | b | c | d |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|



# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

Mientras la cola no este vacia

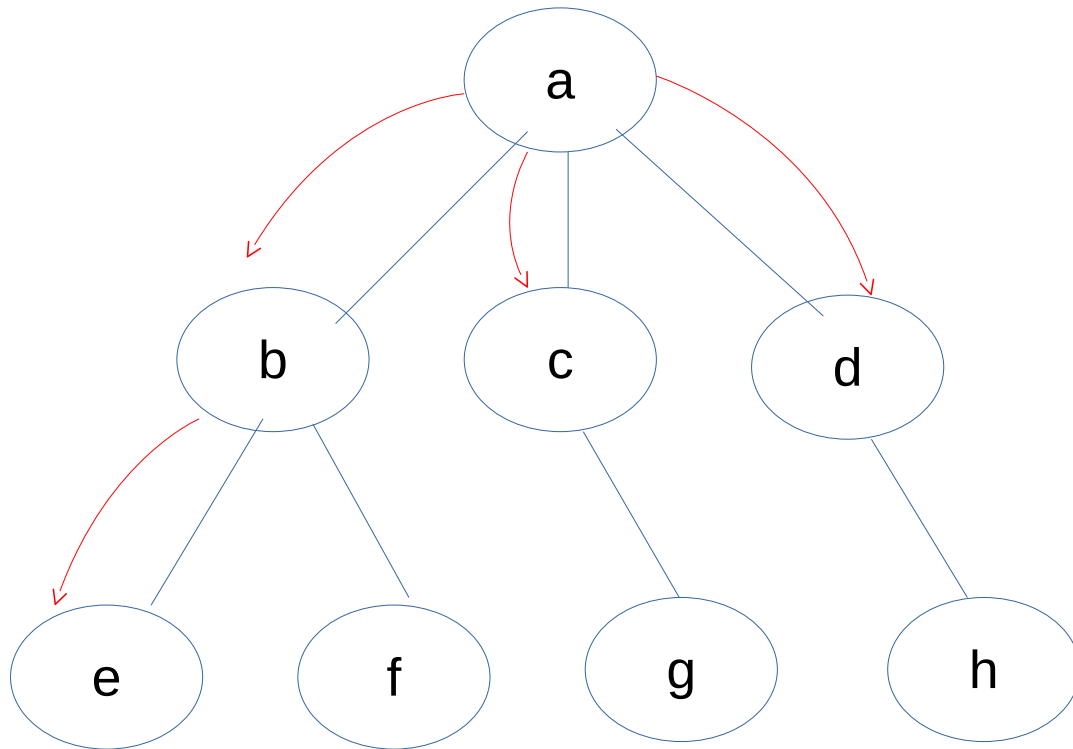
- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

|  |  |  |  |   |   |   |   |  |
|--|--|--|--|---|---|---|---|--|
|  |  |  |  | e | f | g | h |  |
|--|--|--|--|---|---|---|---|--|

|   |   |   |   |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|
| a | b | c | d |  |  |  |  |  |
|---|---|---|---|--|--|--|--|--|

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

Mientras la cola no este vacia

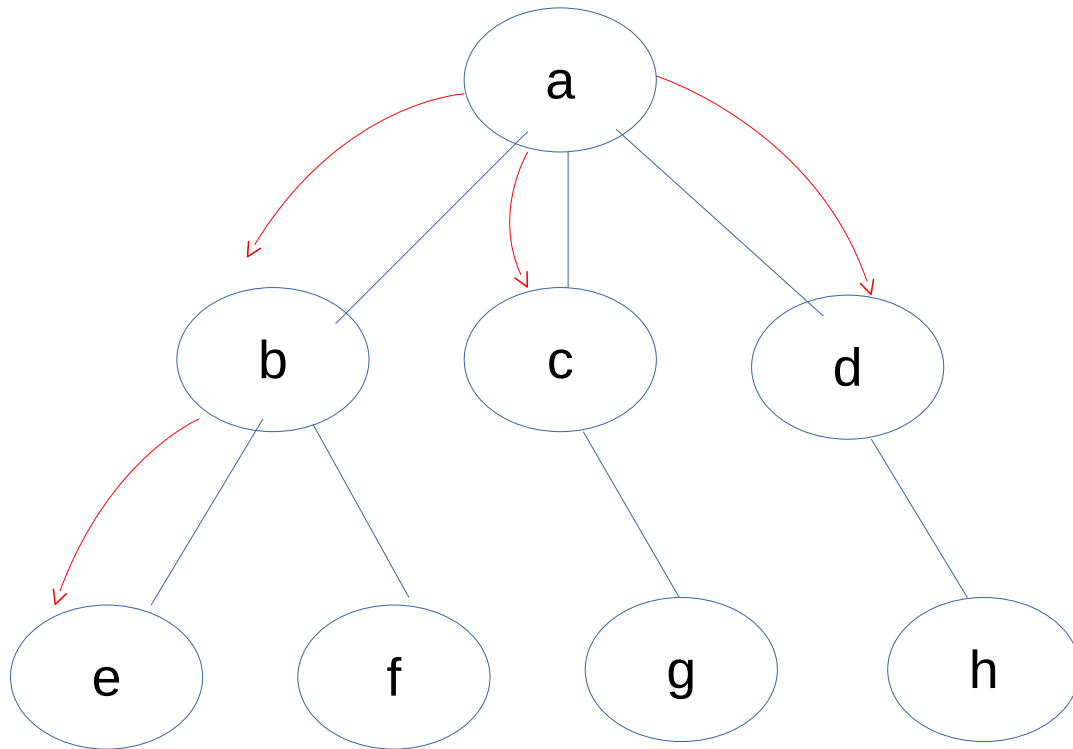
- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

|  |  |  |  |  |   |   |   |  |
|--|--|--|--|--|---|---|---|--|
|  |  |  |  |  | f | g | h |  |
|--|--|--|--|--|---|---|---|--|

|   |   |   |   |   |  |  |  |  |
|---|---|---|---|---|--|--|--|--|
| a | b | c | d | e |  |  |  |  |
|---|---|---|---|---|--|--|--|--|

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

Mientras la cola no este vacia

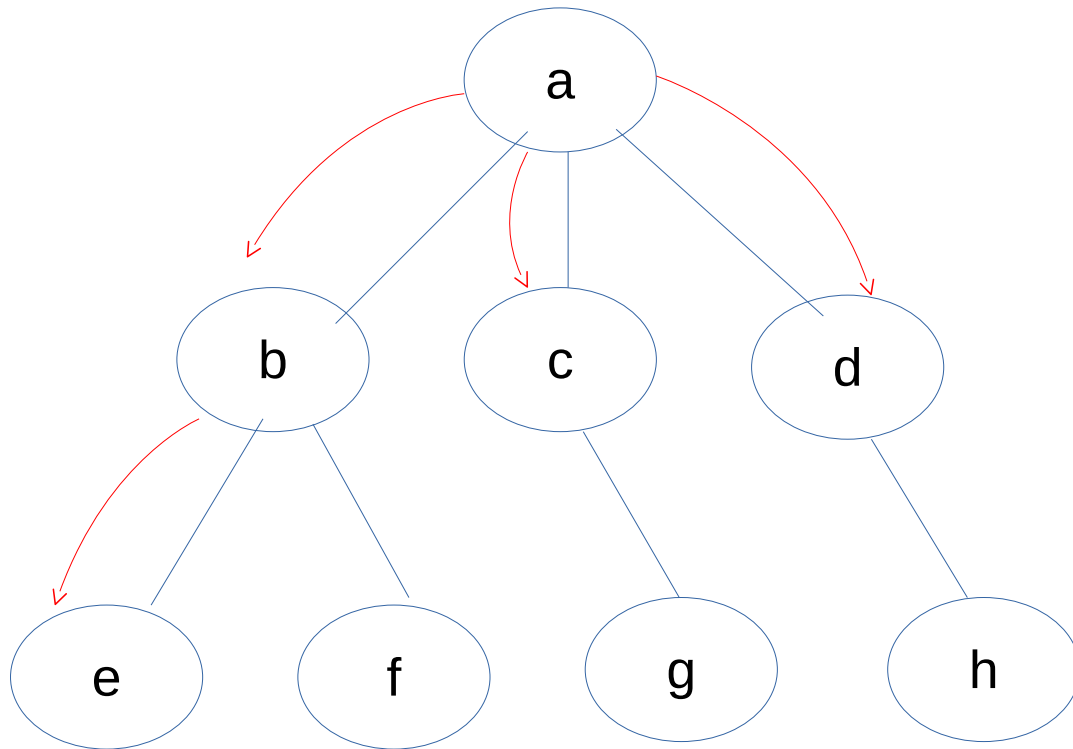
- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

|  |  |  |  |  |  |   |   |  |
|--|--|--|--|--|--|---|---|--|
|  |  |  |  |  |  | g | h |  |
|--|--|--|--|--|--|---|---|--|

|   |   |   |   |   |   |  |  |  |
|---|---|---|---|---|---|--|--|--|
| a | b | c | d | e | f |  |  |  |
|---|---|---|---|---|---|--|--|--|

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

Mientras la cola no este vacia

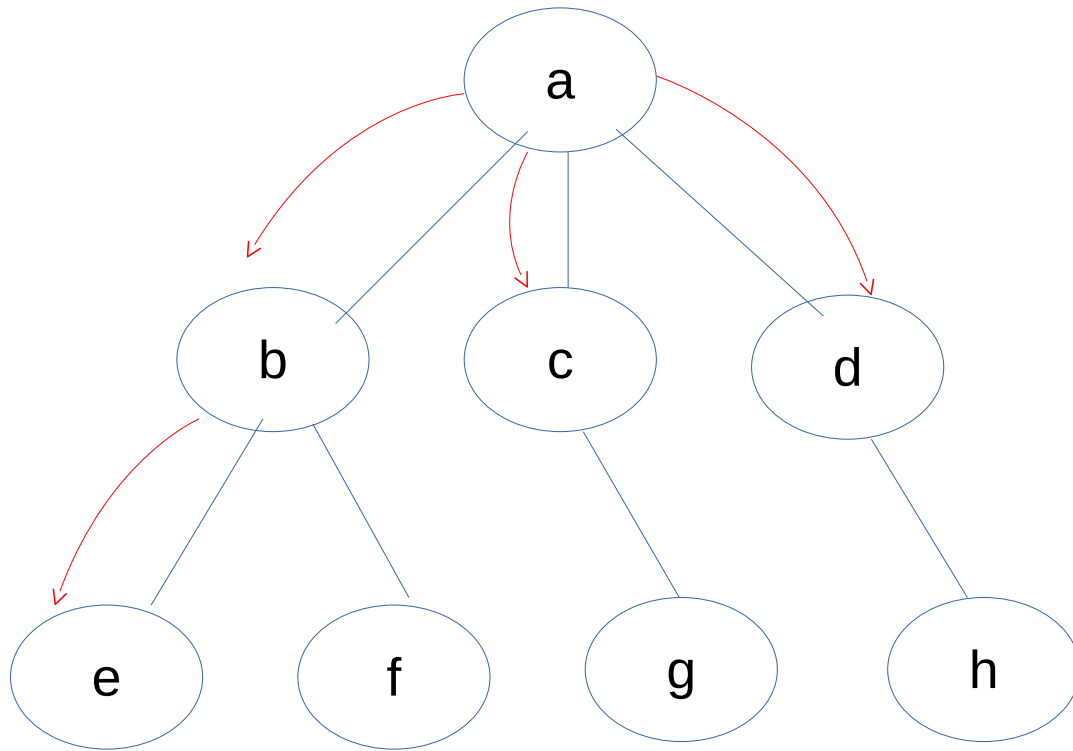
- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

|  |  |  |  |  |  |  |   |  |
|--|--|--|--|--|--|--|---|--|
|  |  |  |  |  |  |  | h |  |
|--|--|--|--|--|--|--|---|--|

|   |   |   |   |   |   |   |  |  |
|---|---|---|---|---|---|---|--|--|
| a | b | c | d | e | f | g |  |  |
|---|---|---|---|---|---|---|--|--|

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

Mientras la cola no este vacia

- Decolar el primer elemento y va al BFS
- Encolar los nodos adyacentes

|  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|
|  |  |  |  |  |  |  |  |  |
|--|--|--|--|--|--|--|--|--|

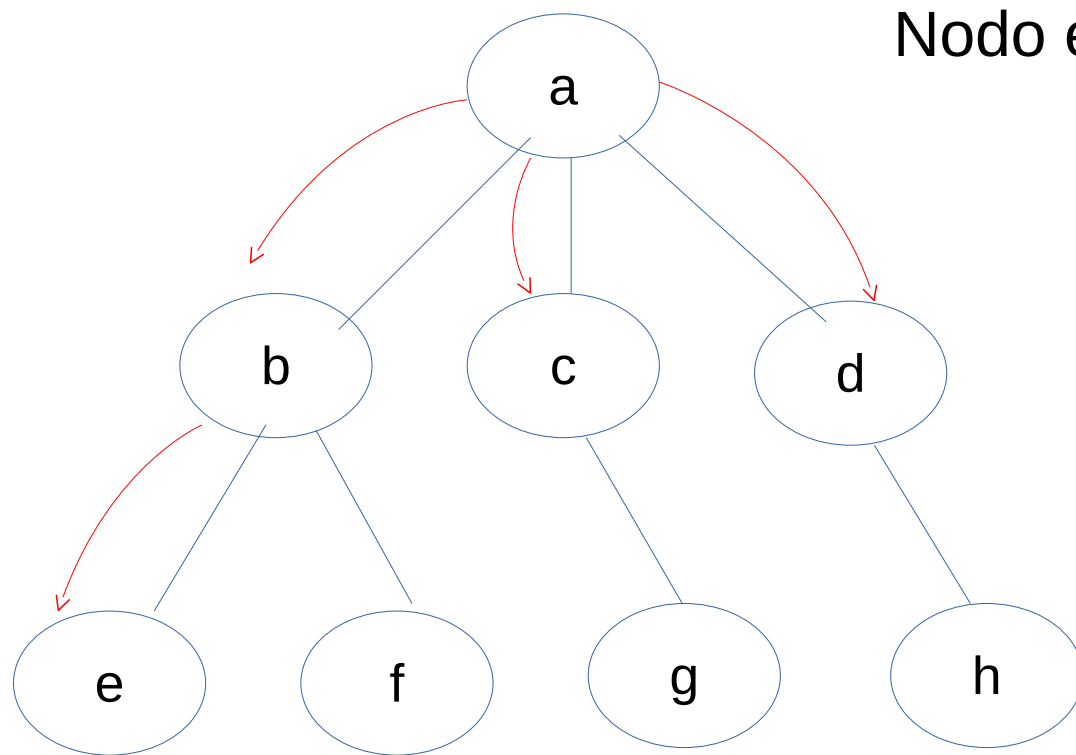
BFS

|   |   |   |   |   |   |   |   |  |
|---|---|---|---|---|---|---|---|--|
| a | b | c | d | e | f | g | h |  |
|---|---|---|---|---|---|---|---|--|

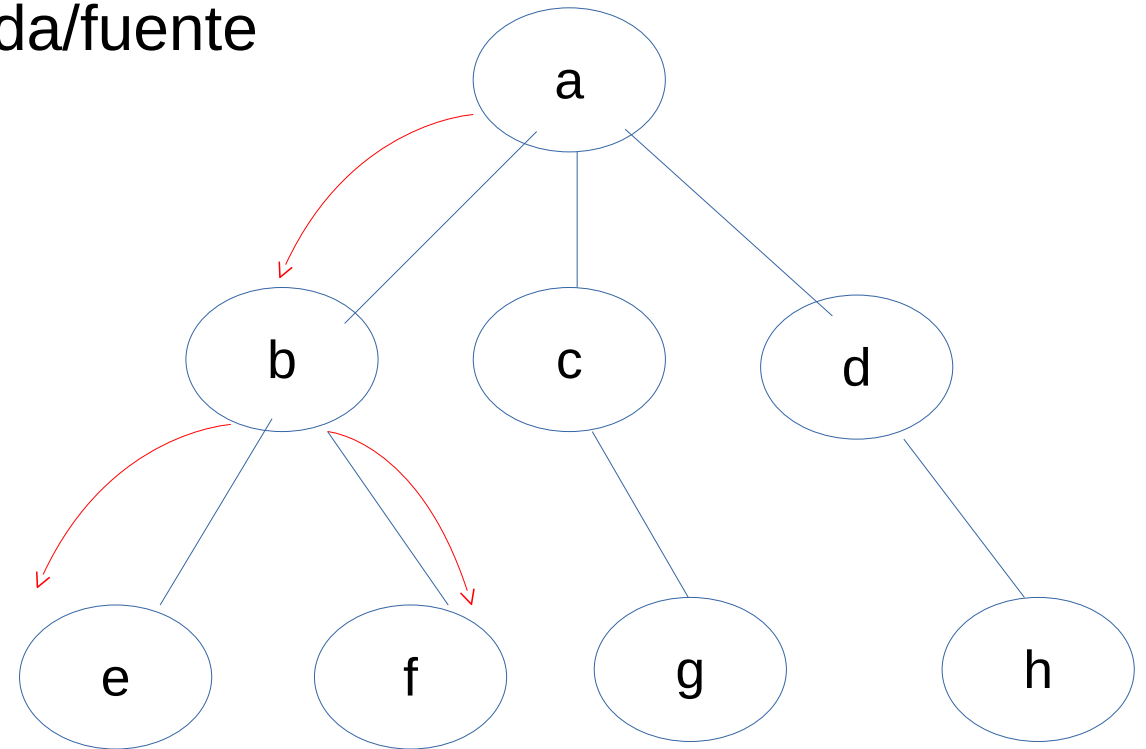
SALIDA BFS: {a, b, c, d, e, f, g, h}

# 7. Recorrido de Grafos

## Diferencias recorridos en Amplitud (BFS) y Profiundidad (DFS)



SALIDA BFS: {a, b, c, d, e, f, g, h}

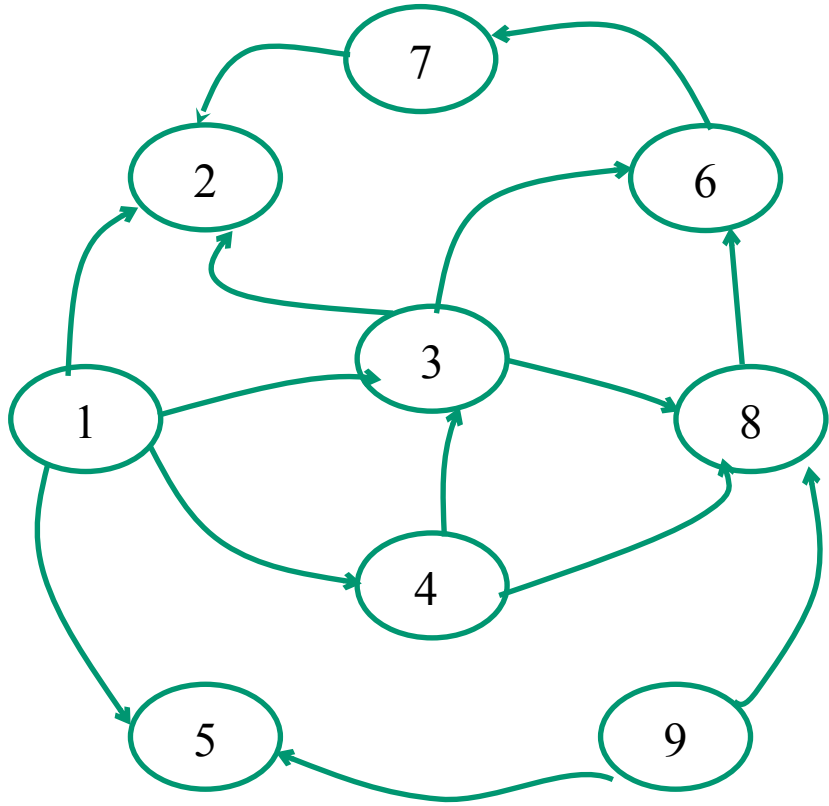


SALIDA DFS: {a, b, e, f, c, g, d, h}

# 7. Recorrido de Grafos

- **Búsqueda (recorrido) en profundidad o e amplitud.** La *expansión* de un grafo dirigido desde un nodo  $v$  se representa po un árbol  $X(v)$  donde:
  - $X(v) = v$ , si  $v$  no tiene suscesores.
  - $X(v) = (v, X(v_1), X(v_2), \dots, X(v_n))$ , si  $v_1, \dots, v_n$  son los sucesores de  $v$ .
- Cuando el grafo tiene ciclos la expansión es infinita.

## 8. Recorrido de Grafos



| Nodos       | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|-------------|---|---|---|---|---|---|---|---|---|
|             | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1, /2,3,4,5 | 1 |   |   |   |   |   |   |   |   |
| 2           |   | 2 |   |   |   |   |   |   |   |
| 3, /2,6,8   |   |   | 3 |   |   |   |   |   |   |
| 6, /7       |   |   |   |   |   | 4 |   |   |   |
| 7, /2       |   |   |   |   |   |   | 5 |   |   |
| 8, /6       |   |   |   |   |   |   |   | 6 |   |
| 4, /3, 8    |   |   |   | 7 |   |   |   |   |   |
| 5, /        |   |   |   |   | 8 |   |   |   |   |
| 9, /5, 8    |   |   |   |   |   |   |   |   | 9 |
| R           | 1 | 2 | 3 | 7 | 8 | 4 | 5 | 6 | 9 |

R. Profundidad: 1, 2, 3, 7, 8, 4, 5, 9

Porque?



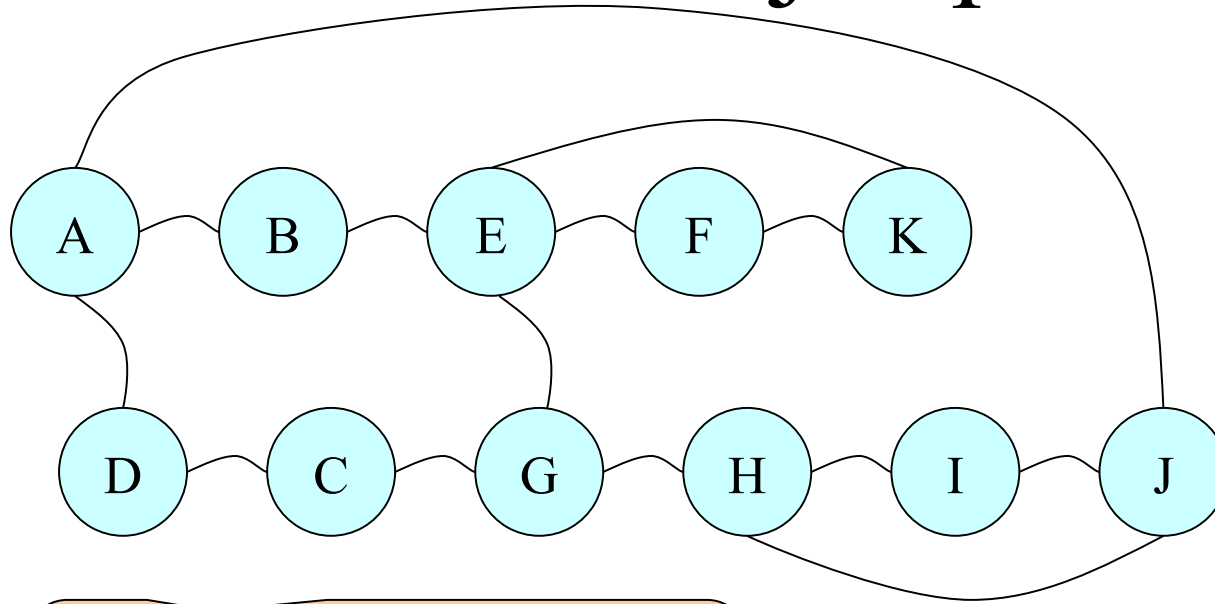
## 8. Recorrido de Grafos

**Búsqueda primero en Profundidad** (DFS: Depth First Search) Algoritmo:

Es una generalización del recorrido Pre Orden de un árbol.

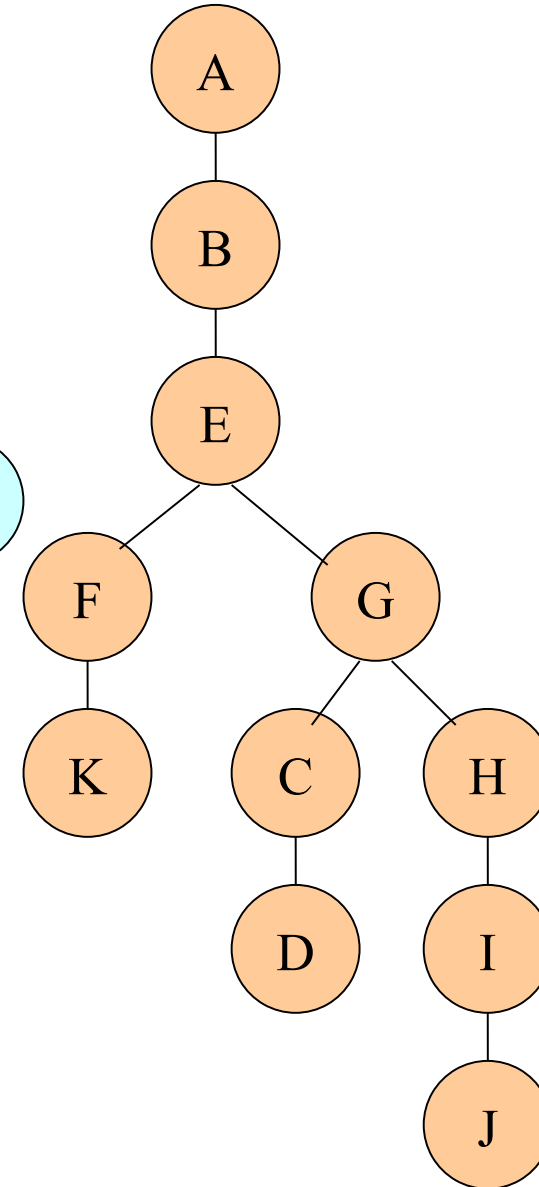
Explora las Aristas del grafo de manera que se visitan los vértices adyacentes al recién visitado, con lo que se consigue “profundizar” en las ramas del grafo

# Ejemplo

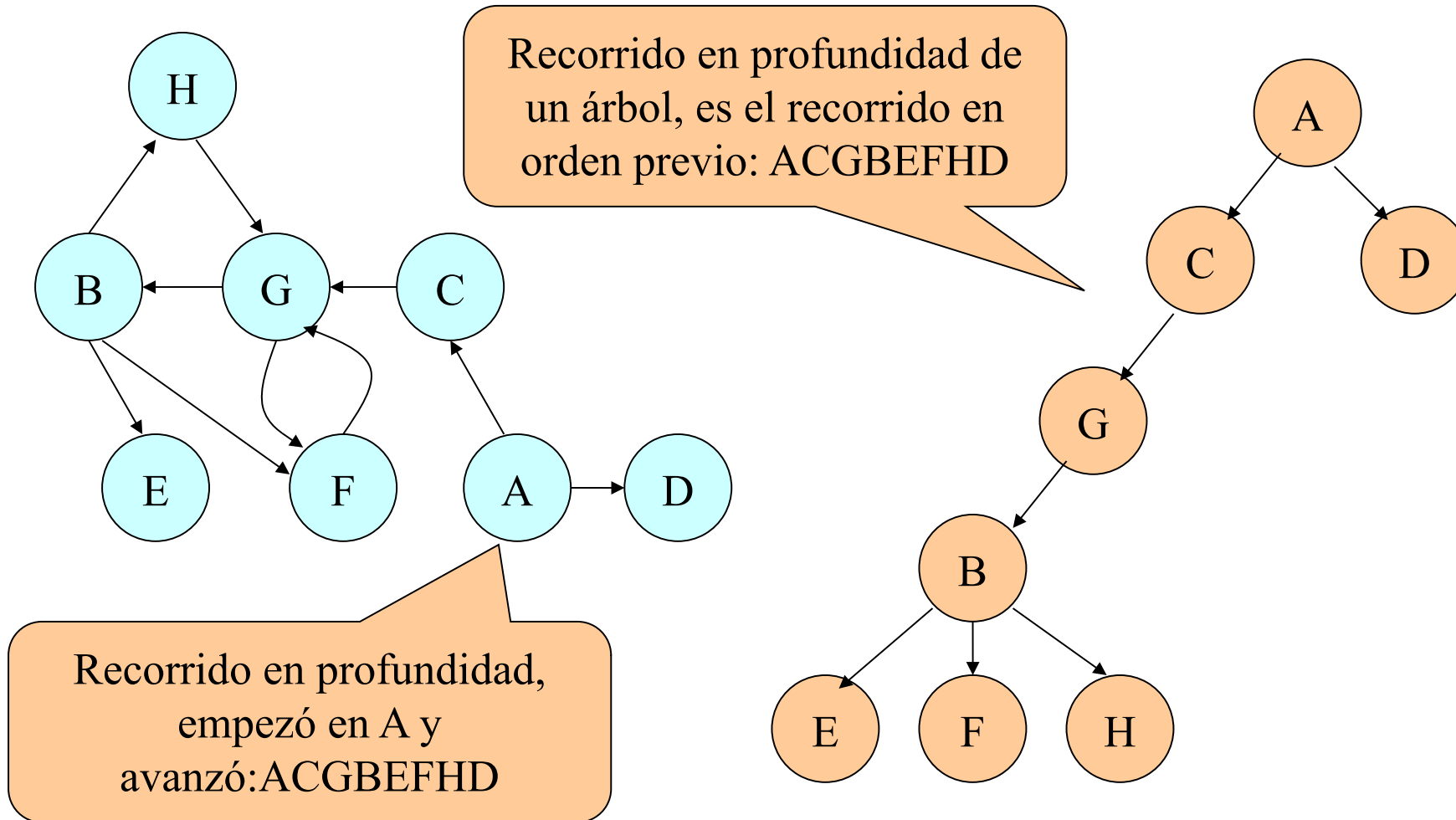


Recorrido en profundidad,  
empezó en A y  
avanzó: ABEFKGCDHIJ

Característica: Después de visitar un nodo, se visitan todos los descendientes del nodo antes que sus hermanos no visitados

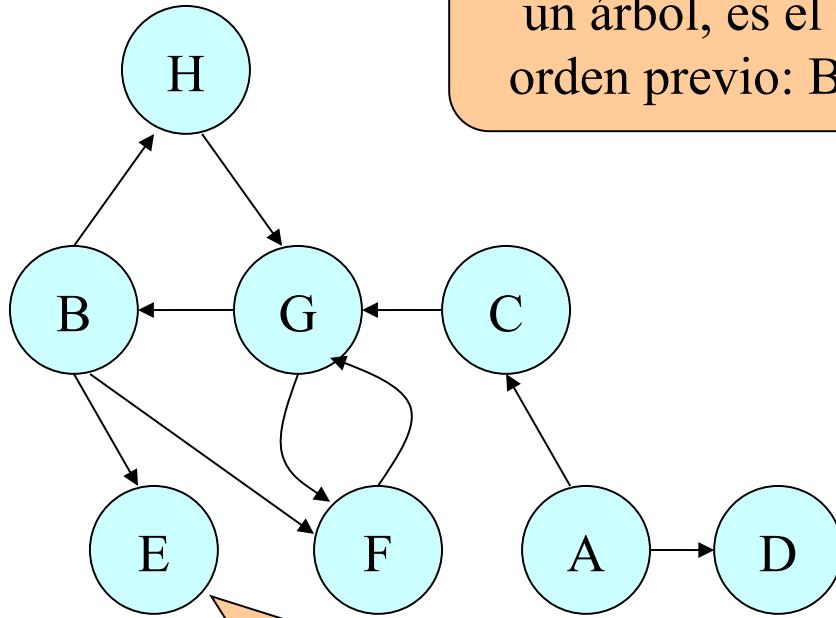


# Ejemplo

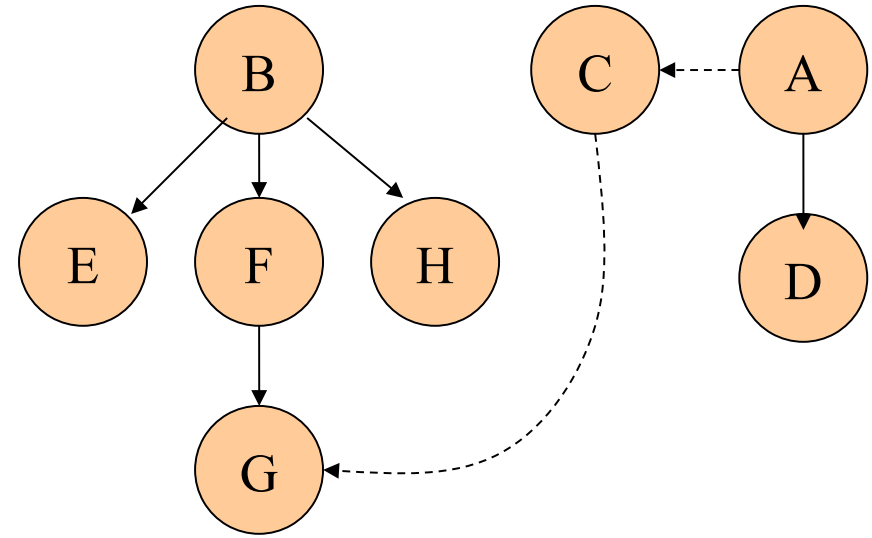


# Ejemplo

Recorrido en profundidad de un árbol, es el recorrido en orden previo: BEFGH CAD

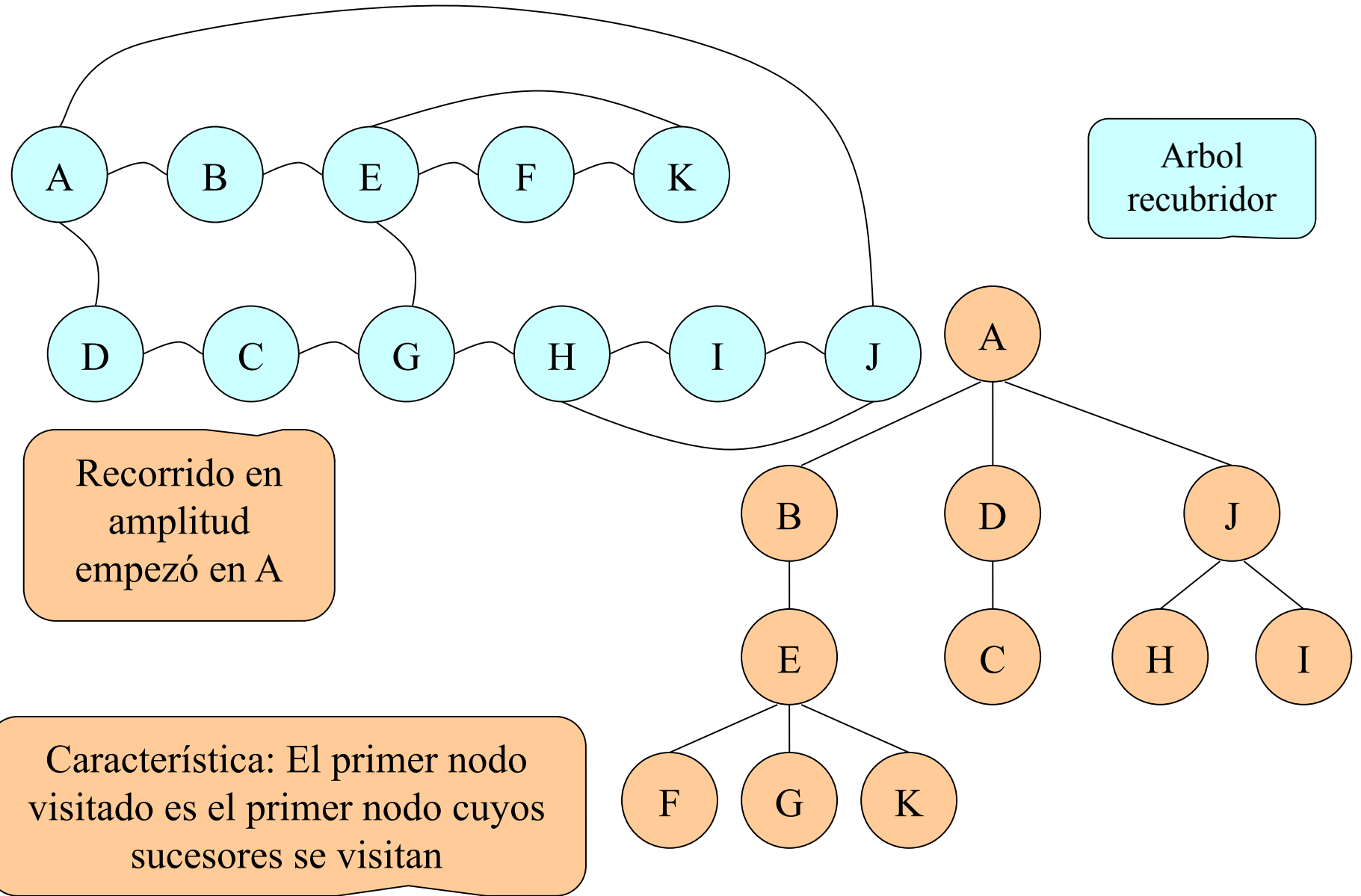


Recorrido en profundidad, empezó en B y avanzó: BEFGHDCAD



¿Puede haber diferentes resultados?

# Ejemplo



¿...?

- **Warshall.** Halla el camino minimo entre todos los vertices
- **Dijkstra.** ¿trayectoria mas corta?
- **Euler.** ¿Paseo euleriano ...?
- **Hamilton.** ¿visitar ciudades ...?
- **Kruskal.** ¿arbol contenido mínimo?
- **Prim** ¿arbol contenido mínimo?
- **Tarjan y Cheriton.** ¿todos contra todos?

## 9. Resumen

- Los grafos pueden modelar problemas simples y complejos
  - Resuelven problemas de circuitos
  - Áreas de ingeniería
  - Dibujo computacional
- Nodos de actores sociales, verificación de posición, centralidad e importancia de cada actor, lo cual permite cuantificar, abstraer relaciones complejas, representando la estructura social de manera gráfica.
- Investigadores: Euler, Dijkstra, Erdos, Harary, Konig, Ringel, Tutte.

# Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- [https://www-geeksforgeeks-org.translate.goog/applications-of-breadth-first-traversal/?\\_x\\_tr\\_sl=en&\\_x\\_tr\\_tl=es&\\_x\\_tr\\_hl=es&\\_x\\_tr\\_pto=tc](https://www-geeksforgeeks-org.translate.goog/applications-of-breadth-first-traversal/?_x_tr_sl=en&_x_tr_tl=es&_x_tr_hl=es&_x_tr_pto=tc)
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesado en Lima, 19.05.2020, 1820 horas], <http://c.conclase.net/ficheros/>
- Silva L. (2010) Estructura de datos y Algoritmos